Повышение квалификации специалистов по информационной безопасности

МЕТОДИЧЕСКИЕ УКАЗАНИЯ
ДЛЯ СТУДЕНТА 4

по курсу
«Методы и принципы проектирования современных защищённых систем. Применение механизмов искусственного интеллекта для анализа данных при построении систем информационной безопасности»

ПОДГОТОВКА ДАННЫХ ДЛЯ АНАЛИТИЧЕСКИХ МОДЕЛЕЙ

Код для последовательного заполнения practice.ipynb

(70 минут)

2026

СОДЕРЖАНИЕ

Как работать с методичкой 4

0. Запуск среды — 5 минут 5

1. Первичный анализ — 8 минут 6

Шаг 1.1. Загрузить исходные таблицы 6

Шаг 1.2. Проверить время и полноту периода 6

Шаг 1.3. Собрать профиль качества 6

2. Очистка и псевдонимизация — 10 минут 8

Шаг 2.1. Добавить функцию токенизации 8

Шаг 2.2. Разделить карантин и рабочие данные 8

Шаг 2.3. Нормализовать поля 8

3. Объекты, признаки и метки — 15 минут 9

Шаг 3.1. Подготовить пятиминутные окна 9

Шаг 3.2. Агрегировать события 9

Шаг 3.3. Развернуть интервалы инцидентов 9

Шаг 3.4. Присоединить метки 10

4. Аудит утечки — 10 минут 11

Шаг 4.1. Создать намеренно недопустимые признаки 11

Шаг 4.2. Заполнить решение по признакам 11

5. Два эксперимента — 17 минут 12

Шаг 5.1. Задать признаки, модель и метрики 12

Шаг 5.2. Выполнить ошибочный эксперимент 12

Шаг 5.3. Создать временные выборки 13

Шаг 5.4. Выполнить корректный эксперимент 13

Шаг 5.5. Построить матрицы ошибок 13

6. Сохранение и проверка — 7 минут 15

Шаг 6.1. Сохранить таблицы 15

Шаг 6.2. Запустить проверку 15
Контрольные выводы 16
Завершение 17

Как работать с методичкой

Откройте `materials/practice.ipynb` и выполняйте разделы строго сверху вниз. Преподаватель сначала объясняет очередной этап, затем вы переносите указанный фрагмент в кодовую ячейку с таким же номером и запускаете её сочетанием `Shift+Enter`. Не работайте в `instructor/solution.ipynb`: это эталонный файл преподавателя.

Под ячейками ряда шагов в notebook находятся готовые контрольные ячейки с заголовком `# Контрольная таблица`. Их не нужно набирать: после своей ячейки запустите контрольную и сверьте выведенную таблицу с экраном преподавателя.

Если переменная не определена, выполните все предыдущие ячейки. Для полного перезапуска выберите **Kernel → Restart Kernel and Run All Cells**.

0. Запуск среды — 5 минут

В терминале перейдите в каталог `materials` и выполните:

```
docker compose up -d --build
docker compose ps
```

Откройте `http://localhost:8888/lab/tree/practice.ipynb`. Выделите первую кодовую ячейку и нажмите ▶ или `Shift+Enter`.

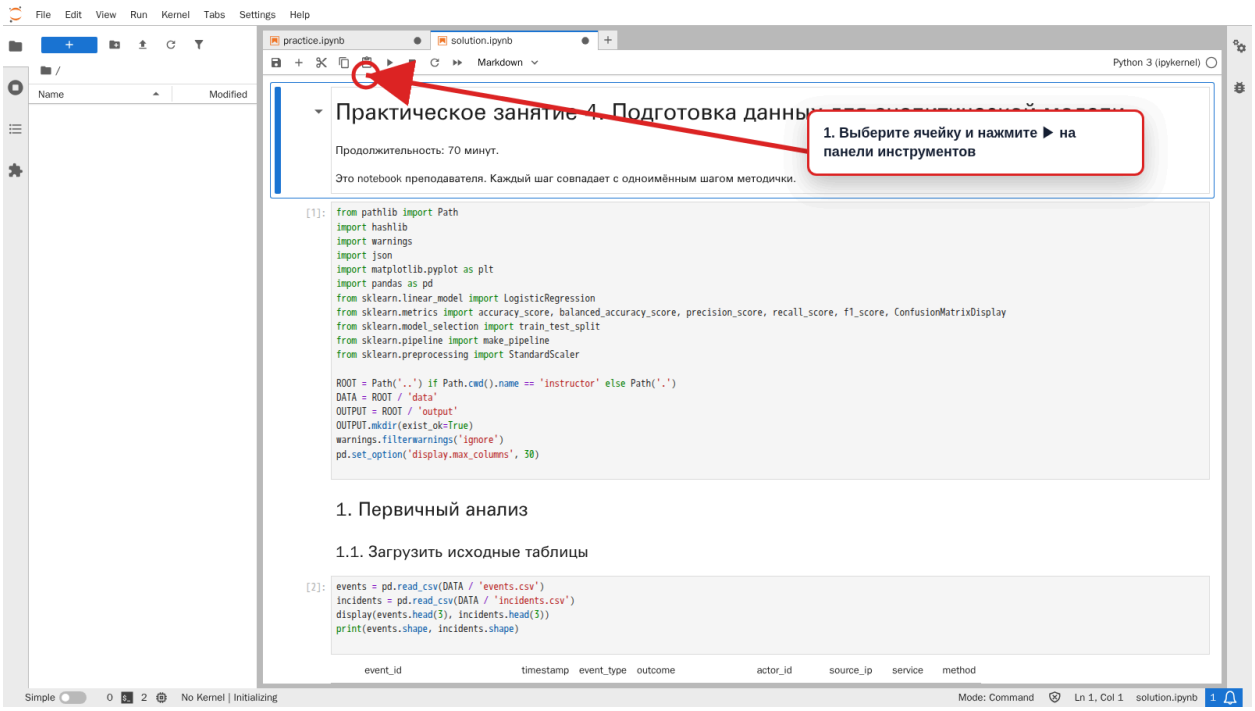


Рисунок 1 — Кнопка запуска ячейки и рабочая область JupyterLab

Импорты и пути уже находятся в шаблоне. Их переписывать не нужно.

1. Первичный анализ — 8 минут

Под заголовком **1. Первичный анализ** находятся три шага. Каждому шагу соответствует отдельный подзаголовок и отдельная кодовая ячейка в `practice.ipynb`. Переносите фрагмент только в ячейку под одноимённым шагом.

Шаг 1.1. Загрузить исходные таблицы

В кодовую ячейку сразу под шагом **1.1** введите:

```
events = pd.read_csv(DATA / 'events.csv')
incidents = pd.read_csv(DATA / 'incidents.csv')
display(events.head(3), incidents.head(3))
print(events.shape, incidents.shape)
```

Ожидаемые размеры: `events — (7991, 8)`, `incidents — (18, 5)`.

Шаг 1.2. Проверить время и полноту периода

```
parsed_time = pd.to_datetime(events['timestamp'], utc=True, errors='coerce')
observed_hours = pd.DatetimeIndex(
    parsed_time.dropna().dt.floor('h').unique()
)
expected_hours = pd.date_range(
    '2026-07-01', periods=56 * 24, freq='h', tz='UTC'
)
```

Шаг 1.3. Собрать профиль качества

```
quality = {
    'records': len(events),
    'missing_source': int(events['source_ip'].isna().sum()),
    'invalid_outcome': int(
        (~events['outcome'].isin(['success', 'failure'])).sum()
    ),
    'duplicates': int(events['event_id'].duplicated().sum()),
    'noncanonical_type': int(
        (events['event_type'] != events['event_type'].str.lower()).sum()
    ),
    'non_utc': int(
        (~events['timestamp'].str.endswith('+00:00')).sum()
    ),
    'gap_hours': int(
        len(expected_hours.difference(observed_hours))
    ),
}
quality_summary = pd.DataFrame({
```

```
    'check': quality.keys(),
    'value': quality.values(),
})
quality_summary
```

Сверьте результат: 7 991 запись; 6 строк без источника; 4 недопустимых результата; 8 дублей; 13 ненормализованных типов; 24 отметки не в UTC; 3 отсутствующих часа.

2. Очистка и псевдонимизация — 10 минут

В этом разделе также используется отдельная кодовая ячейка для каждого шага.

Шаг 2.1. Добавить функцию токенизации

Сначала перенесите в неё функцию:

```
def token(prefix, value):
    digest = hashlib.sha256(
        f'traceshop-practice-4:{value}'.encode()
    ).hexdigest()[:10]
    return f'{prefix}-{digest}'
```

Шаг 2.2. Разделить карантин и рабочие данные

Продолжите в той же ячейке:

```
invalid = (
    events['source_ip'].isna()
    | ~events['outcome'].isin(['success', 'failure'])
)
quarantine = events.loc[invalid].copy()
clean = (
    events.loc[~invalid]
    .drop_duplicates('event_id')
    .copy()
)
```

Шаг 2.3. Нормализовать поля

```
clean['timestamp'] = pd.to_datetime(clean['timestamp'], utc=True)
clean['event_type'] = clean['event_type'].str.lower()
clean['source_id'] = clean['source_ip'].map(
    lambda value: token('src', value)
)
clean['actor_id'] = clean['actor_id'].map(
    lambda value: token('user', value)
)
print('карантин:', len(quarantine),
      'чистых записей:', len(clean))
```

Контрольная строка: `карантин: 10 чистых записей: 7973`.

3. Объекты, признаки и метки — 15 минут

Шаг 3.1. Подготовить пятиминутные окна

```
clean['window_start'] = clean['timestamp'].dt.floor('5min')
clean['is_failure'] = (
    clean['outcome'] == 'failure'
).astype(int)
clean['is_success'] = (
    clean['outcome'] == 'success'
).astype(int)
```

Шаг 3.2. Агрегировать события

```
objects = (
    clean.groupby(['source_id', 'window_start'], as_index=False)
    .agg(
        attempt_count=('event_id', 'size'),
        failure_count=('is_failure', 'sum'),
        success_count=('is_success', 'sum'),
        unique_accounts=('actor_id', 'nunique'),
    )
)
objects['failure_rate'] = (
    objects['failure_count'] / objects['attempt_count']
)
objects['night_activity'] = (
    objects['window_start'].dt.hour
    .isin([0, 1, 2, 3, 4, 5])
    .astype(int)
)
objects['object_id'] = [
    token('obj', f'{source}:{window.isoformat()}')
    for source, window
    in zip(objects['source_id'], objects['window_start'])
]
```

Шаг 3.3. Развернуть интервалы инцидентов

```
incidents['start'] = pd.to_datetime(incidents['start'], utc=True)
incidents['end'] = pd.to_datetime(incidents['end'], utc=True)
incidents['source_id'] = incidents['source_ip'].map(
    lambda value: token('src', value)
)

label_rows = []
for row in incidents.itertuples():
    windows = pd.date_range(
        row.start.floor('5min'),
        row.end.floor('5min'),
        freq='5min',
    )
    for window in windows:
```

```
        label_rows.append({
            'source_id': row.source_id,
            'window_start': window,
            'scenario_id': row.scenario_id,
            'attack_type': row.attack_type,
        })
labels = pd.DataFrame(label_rows)
```

Шаг 3.4. Присоединить метки

```
objects = objects.merge(
    labels,
    how='left',
    on=['source_id', 'window_start'],
)
objects['scenario_id'] = objects['scenario_id'].fillna('')
objects['label'] = objects['attack_type'].fillna('normal')
objects['is_attack'] = (
    objects['label'] != 'normal'
).astype(int)
objects.drop(columns='attack_type', inplace=True)
print('объектов:', len(objects))
print(objects['is_attack'].value_counts())
```

Должно получиться 6 790 объектов, из них 27 атакующих.

4. Аудит утечки — 10 минут

Шаг 4.1. Создать намеренно недопустимые признаки

```
success_times = (
    clean.loc[clean['is_success'].eq(1)]
    .groupby('source_id')['timestamp']
    .apply(list)
    .to_dict()
)

def future_success(row):
    begin = row.window_start + pd.Timedelta(minutes=5)
    end = row.window_start + pd.Timedelta(minutes=15)
    return int(any(
        begin <= timestamp < end
        for timestamp in success_times.get(row.source_id, [])
    ))

objects['future_success_10m'] = objects.apply(
    future_success, axis=1
)
objects['incident_confirmed'] = objects['is_attack']
```

Шаг 4.2. Заполнить решение по признакам

```
feature_audit = pd.DataFrame([
    ['attempt_count', 'события текущего окна', 'оставить'],
    ['failure_rate', 'события текущего окна', 'оставить'],
    ['unique_accounts', 'события текущего окна', 'оставить'],
    ['night_activity', 'время текущего окна', 'оставить'],
    ['future_success_10m',
     'события после момента решения', 'исключить'],
    ['incident_confirmed',
     'результат последующего расследования', 'исключить'],
    ['scenario_id',
     'служебный идентификатор разметки', 'исключить'],
], columns=['feature', 'available_when', 'decision'])
feature_audit
```

Критерий: поле разрешено, только если оно существует в момент принятия решения.

5. Два эксперимента — 17 минут

Шаг 5.1. Задать признаки, модель и метрики

```
SAFE = [
    'attempt_count', 'failure_rate',
    'unique_accounts', 'night_activity',
]
LEAKY = SAFE + ['future_success_10m', 'incident_confirmed']

def model():
    return make_pipeline(
        StandardScaler(),
        LogisticRegression(
            class_weight='balanced',
            max_iter=1000,
            random_state=404,
        ),
    )

def scores(name, truth, predicted):
    return {
        'experiment': name,
        'accuracy': accuracy_score(truth, predicted),
        'balanced_accuracy':
            balanced_accuracy_score(truth, predicted),
        'precision': precision_score(
            truth, predicted, zero_division=0),
        'recall': recall_score(
            truth, predicted, zero_division=0),
        'f1': f1_score(truth, predicted, zero_division=0),
    }
```

Шаг 5.2. Выполнить ошибочный эксперимент

```
wrong_train, wrong_test = train_test_split(
    objects,
    test_size=0.25,
    stratify=objects['is_attack'],
    random_state=404,
)
wrong_model = model().fit(
    wrong_train[LEAKY],
    wrong_train['is_attack'],
)
wrong_prediction = wrong_model.predict(wrong_test[LEAKY])
wrong_metrics = scores(
    'случайное разбиение + утечка',
    wrong_test['is_attack'],
    wrong_prediction,
)
```

Шаг 5.3. Создать временные выборки

```
first_day = pd.Timestamp('2026-07-01', tz='UTC')
objects['day'] = (
    objects['window_start'].dt.floor('D') - first_day
).dt.days + 1

objects['split'] = 'gap'
objects.loc[objects['day'].le(34), 'split'] = 'train'
objects.loc[
    objects['day'].between(36, 45), 'split'
] = 'validation'
objects.loc[objects['day'].ge(47), 'split'] = 'test'

train = objects.loc[objects['split'].eq('train')].copy()
validation = objects.loc[
    objects['split'].eq('validation')
].copy()
test = objects.loc[objects['split'].eq('test')].copy()
```

Шаг 5.4. Выполнить корректный эксперимент

```
correct_model = model().fit(
    train[SAFE],
    train['is_attack'],
)
correct_prediction = correct_model.predict(test[SAFE])
correct_metrics = scores(
    'временное разбиение без утечки',
    test['is_attack'],
    correct_prediction,
)
metrics = pd.DataFrame([
    wrong_metrics, correct_metrics
]).round(3)
metrics
```

Контрольные F1: `1.000` для ошибочного эксперимента и `0.333` для корректного.

Шаг 5.5. Построить матрицы ошибок

```
cell_names = [
    ['TN\nверная норма', 'FP\nложная тревога'],
    ['FN\nпропуск атаки', 'TP\nнайдена атака'],
]
experiments = [
    ('Случайное разбиение и утечка',
     wrong_test['is_attack'], wrong_prediction),
    ('Временное разбиение',
     test['is_attack'], correct_prediction),
]
fig, axes = plt.subplots(1, 2, figsize=(10, 4))
```

```
for ax, (title, truth, predicted) in zip(axes, experiments):
    ConfusionMatrixDisplay.from_predictions(
        truth, predicted, ax=ax, colorbar=False,
    )
    for row in range(2):
        for col in range(2):
            ax.text(
                col, row - 0.30, cell_names[row][col],
                ha='center', va='center',
                fontsize=10, fontweight='bold',
                bbox=dict(boxstyle='round', facecolor='white'),
            )
    ax.set_title(title)
    ax.set_xlabel('Прогноз модели')
    ax.set_ylabel('Истинный ответ')
plt.tight_layout()
```

В корректном эксперименте: 4 найденные атаки, 2 пропуска и 14 ложных тревог.

6. Сохранение и проверка — 7 минут

Шаг 6.1. Сохранить таблицы

Перечень экспортируемых столбцов уже указан в студенческом шаблоне — повторно набирать его не требуется. Допишите вызовы:

```
quality_summary.to_csv(
    OUTPUT / 'quality-summary.csv', index=False)
quarantine.to_csv(OUTPUT / 'quarantine.csv', index=False)
objects[export_columns].to_csv(
    OUTPUT / 'objects.csv', index=False)
train[export_columns].to_csv(OUTPUT / 'train.csv', index=False)
validation[export_columns].to_csv(
    OUTPUT / 'validation.csv', index=False)
test[export_columns].to_csv(OUTPUT / 'test.csv', index=False)
feature_audit.to_csv(
    OUTPUT / 'feature-audit.csv', index=False)
metrics.to_csv(OUTPUT / 'metrics.csv', index=False)
```

Создайте паспорт по заготовке в ячейке и запишите его:

```
(OUTPUT / 'dataset-card.md').write_text(
    card, encoding='utf-8'
)
```

Шаг 6.2. Запустить проверку

В последней ячейке выполните:

```
!python3 verify/verify.py output
```

Успешное завершение содержит пять строк с ✓ и сообщение `Результат принят.` Если проверка не прошла, исправьте указанный этап, повторно выполните ячейку сохранения и запустите проверку ещё раз.

Контрольные выводы

– Профиль качества рассчитывают до очистки, иначе масштаб дефектов скрывается.
– Карантин сохраняет недостоверные строки для анализа, но исключает их из модели.
– Единица наблюдения задаётся до вычисления признаков.
– Доступность признака в момент решения важнее его высокой корреляции с целью.
– Случайное разбиение временных данных и утечка дают неправдоподобно идеальную оценку.
– Для редкого класса Accuracy рассматривается вместе с Precision, Recall, F1 и матрицей ошибок.

Завершение

Notebook сохраняется автоматически. После занятия остановите среду из каталога `materials`:

```
docker compose down
```